

1. What This PDF Covers

This document explains the technical side in four simple parts:

- **Technology stack** - frontend, backend, database, cache, queues, authentication, storage, testing and deployment.
- **Where each technology is used** - for example, what Next.js does, what NestJS does, what PostgreSQL stores and what Redis does.
- **Main architecture and folder structure** - how developers should separate the Attendance code so it remains maintainable.
- **Final technical flow** - how data moves from enrollment and calendar creation to attendance capture, processing, final results, reports and AI.

ONE COMMON ATTENDANCE PLATFORM -> TENANT CONFIGURATION -> UNIVERSITY / ACADEMY / BOOTCAMP / CORPORATE

2. Complete Recommended Technology Stack

Area	Technology	Main use in Attendance
Frontend	Next.js 16 + React + TypeScript	Attendance screens for Admin, Faculty/Trainer and Learner.
Backend	Node.js + NestJS + TypeScript	APIs, validation, authorization, attendance business workflow and integrations.
Database	PostgreSQL	Permanent transactional data: enrollments, sessions, evidence, attendance records, policies and audit.
ORM	Prisma	Typed backend access to PostgreSQL and controlled database migrations.
Cache	Redis	Short-lived/high-speed data such as QR state, rate limits and frequently read session information.
Authentication	Existing LMS SSO / OIDC / JWT	Identifies the logged-in user, tenant, role and scope.
File storage	Existing S3-compatible storage	Leave/correction attachments and report/export files.
API contract	REST + OpenAPI / Swagger	Frontend/backend communication and external integration documentation.
Testing	Jest/Vitest + Playwright	Business-rule, API and complete browser-flow testing.
Observability	OpenTelemetry + existing logging/metrics	Detects API, queue, integration and attendance-processing failures.
Deployment	Docker + existing CI/CD; Kubernetes only if already used	Consistent application packaging and production deployment.

3. Frontend - Next.js 16

3.1 What Next.js is used for

Next.js is the web application framework. It is responsible for the pages/routes that users open inside NeuroLXP. Attendance business decisions should not be calculated inside these pages; the frontend calls the backend APIs.

- Attendance Dashboard.
- Today's Sessions.
- Take Attendance roster.
- Dynamic QR display for Faculty/Trainer.
- Learner check-in screen.
- My Attendance and subject/course percentage.
- Leave and correction request screens.
- Reports, filters and policy/settings screens.

3.2 Why Next.js fits

NeuroLXP is already working with Next.js. Keeping Attendance in the same frontend technology avoids another separate frontend application and lets Attendance reuse the existing layout, authentication, navigation and design system.

4. React - What It Is Used For

React builds reusable interactive UI components inside the Next.js application.

Component example	What React handles
SessionRoster	Displays learners and updates attendance status controls.
AttendanceStatusControl	Present / Late / Absent / Excused selection.
BulkAttendanceToolbar	Mark all Present and edit exceptions.
DynamicQrPanel	Displays/refetches the current short-lived QR.
AttendanceSummary	Shows totals and percentage.
RiskBanner	Shows low-attendance warning.
CorrectionRequestForm	Learner submits a correction request.
AttendanceReportFilters	Filters by tenant scope, course, batch, date and status.

Rule: React should display and collect information. Official attendance rules belong in the backend.

5. TypeScript - Why We Use It

TypeScript adds strong types to both frontend and backend JavaScript. Attendance contains many important states and identifiers, so types reduce accidental mistakes.

- Shared types for Present, Late, Absent, Partial, Excused and Pending.
- Shared tenant types: University, Skilling Academy, Bootcamp and Corporate.
- Typed API request/response contracts.
- Safer refactoring when attendance rules grow.
- Fewer errors such as sending a course ID where a session ID is expected.

6. Backend - Node.js

Node.js is the runtime that executes the backend TypeScript/JavaScript services. It is well suited for API-heavy and event/integration-heavy LMS workflows.

- Receives frontend API requests.
- Receives QR check-ins and integration webhooks.
- Calls database/cache/queue services.
- Runs NestJS application services.
- Supports real-time/high-concurrency I/O such as many learners checking in around class start time.

7. Backend Framework - NestJS

7.1 What NestJS is used for

NestJS gives the backend a clean modular structure. Instead of putting all Attendance logic in one large file, each business area becomes a module.

NestJS concept	Attendance use
----------------	----------------

Controller	Receives HTTP requests such as open session, mark attendance or learner check-in.
Service	Runs application/business workflow.
Guard	Checks login, tenant, role and scope before allowing an action.
DTO / validation	Checks that API input is valid.
Module	Groups related features such as Attendance, Enrollment or Calendar.
Interceptor/filter	Standard logging, tracing and error handling.
Swagger integration	Generates API documentation.

7.2 Example

NEXT.JS 'TAKE ATTENDANCE' -> NESTJS CONTROLLER -> AUTH/TENANT GUARD -> ATTENDANCE SERVICE -> POLICY SERVICE -> DATABASE

For example, when Faculty clicks Submit, the NestJS controller receives the request. Guards verify the Faculty belongs to the correct tenant and session scope. The Attendance service validates the session, finalizes records, and then triggers downstream events.

8. PostgreSQL - What It Is Used For

PostgreSQL is the main permanent source of truth. If the server restarts or Redis is cleared, official Attendance data must still exist in PostgreSQL.

Data area	Examples stored in PostgreSQL
Tenant/organization references	Tenant ID, department/center/team references.
Enrollment	Learner, course/group, effective dates and enrollment status.
Calendar/session	Session date, start/end, timezone, faculty, group and lifecycle status.
Attendance policy	Allowed methods, grace period, threshold and policy version.
Roster snapshot	Who was eligible for a finalized session.
Evidence	Manual mark, QR receipt, virtual join/leave or external event.
Official record	Learner + session + final attendance status.
Corrections/leave	Request, decision, reason and resulting change.
Audit	Who changed what, when and why.

PostgreSQL is not used just for reports. It is the authoritative transactional store for the Attendance system.

9. Prisma - What It Is Used For

Prisma is the ORM/data-access layer between NestJS and PostgreSQL. It lets developers work with typed models and migrations instead of writing raw SQL everywhere.

- Defines application database models.
- Reads/writes Attendance data from NestJS services.
- Creates controlled schema migrations as features evolve.
- Provides transactions when several related records must change together.

If the existing NeuroLXP backend already uses another mature ORM, use that existing standard rather than introducing Prisma only for Attendance.

10. Redis - What It Is Used For

Redis is fast temporary storage. It should support the system, but it should not become the only copy of official Attendance.

Redis use	Example
Dynamic QR state	Short-lived token/nonce or related session state.
Rate limiting	Prevent excessive check-in attempts.
Cache	Frequently requested session/summary data.
Distributed locks	Avoid two workers processing the same critical operation where needed.
Queue backend	BullMQ stores job state in Redis.

Anything that must survive cache loss belongs in PostgreSQL

11. Main Architecture

NEXT.JS WEB UI

-> NESTJS API

-> AUTH + TENANT/SCOPE GUARDS

-> DOMAIN MODULES

-> POSTGRESQL + REDIS

-> BULLMQ WORKERS / NOTIFICATIONS / INTEGRATIONS / ANALYTICS

The UI never talks directly to PostgreSQL or Redis. It calls authenticated backend APIs. The backend owns validation, tenant isolation and official Attendance processing.

12. Main Architecture Folder Structure

This is the recommended maintainable structure. It is shown as architecture documentation - not as a downloadable folder package.

```
Calendar
|
+-- apps/
|   +-- web/                                # Next.js frontend
|   |   +-- app/
|   |   |   +-- attendance/                # Attendance routes/pages
|   |   |   +-- components/
|   |   |   +-- attendance/                # Reusable React components
|   |   |   +-- lib/
|   |   |   +-- attendance-api/            # Frontend API client
|   |   |
|   |   +-- api/                            # NestJS backend
|   |   |   +-- src/modules/
|   |   |   |   +-- tenancy/                # Tenant context/isolation
|   |   |   |   +-- organizations/          # Department/center/team
|   |   |   |   +-- catalog/                # Course/program/subject/training
|   |   |   |   +-- enrollments/           # Learner eligibility
|   |   |   |   +-- calendars/             # Terms/holidays
|   |   |   |   +-- sessions/              # Timetable + sessions
|   |   |   |   +-- attendance/            # Capture/review/final records
|   |   |   |   +-- policies/              # Attendance rule engine
|   |   |   |   +-- notifications/         # Alerts/reminders
|   |   |   |   +-- reports/               # Aggregation/report APIs
|   |   |   |   +-- integrations/          # SIS/HRMS/Teams/Zoom adapters
|   |   |   |   +-- audit/                 # Change history
|   |   |
|   |   +-- worker/                         # BullMQ workers
|   |   |   +-- jobs/
|   |   |   |   +-- generate-sessions
|   |   |   |   +-- refresh-rosters
|   |   |   |   +-- close-attendance
```

```

|         +-- reconcile-events
|         +-- aggregate-results
|         +-- send-notifications
|
+-- packages/
|   +-- shared/           # Shared TypeScript types/contracts
|   +-- ui/               # Shared UI/design-system pieces
|   +-- config/           # Shared TS/lint/build config
|
+-- database/
|   +-- prisma/           # Schema and migrations
|
+-- tests/
|   +-- e2e/              # Playwright flows
|
+-- infra/
|   +-- docker/           # Container definitions
|   +-- deployment/       # Existing platform deployment config

```

13. What Each Main Folder Owns

Folder/module	Responsibility
web/attendance	Screens and route-level UI only.
components/attendance	Reusable React Attendance components.

Folder/module	Responsibility
api/tenancy	Trusted tenant context and tenant isolation.
api/organizations	Department, center and team hierarchy references.
api/catalog	Program/course/subject/training references.
api/enrollments	Who is eligible for attendance.
api/calendars	Academic/training calendar and holidays.
api/sessions	Timetable, schedule, session generation and lifecycle.
api/attendance	Evidence capture, roster status, review, submit and official record.
api/policies	Deterministic Present/Late/Absent/Partial/Excused rules.
api/notifications	Attendance reminders and outcome alerts.
api/reports	Attendance summaries, dashboards and exports.
api/integrations	Provider adapters; keep Teams/Zoom/SIS/HRMS code out of core rules.
api/audit	Immutable business/security change history.
worker/jobs	Time-based and asynchronous processing.
packages/shared	Common TypeScript enums/types/API contracts.
database/prisma	Schema and migration history.
tests/e2e	Complete role-based multi-tenant flows.

14. Why This Folder Structure Is Important

- University, Academy, Bootcamp and Corporate do not get four separate Attendance folders.
- Tenant terminology belongs in configuration and presentation.
- Core attendance rules stay in one policy engine.
- Provider-specific integration code stays behind adapters.
- React components do not contain official attendance calculations.
- Background jobs are separate from synchronous API controllers.

- Database migrations and tests remain visible and maintainable.

TENANT -> ORGANIZATION -> COURSE/PROGRAM/TRAINING -> SECTION/BATCH/COHORT/AUDIENCE -> USERS

15. Technical Flow - Step 1: Existing LMS Master Data

Attendance consumes or references these existing LMS masters. It should not duplicate them unnecessarily.

ENROLLMENT CREATED/IMPORTED -> NESTJS ENROLLMENT SERVICE -> POSTGRESQL -> ELIGIBILITY AVAILABLE

16. Technical Flow - Step 2: Enrollment

Enrollment tells Attendance which learner is expected in which learning group and during which dates.

ADMIN UI -> NEXT.JS -> NESTJS CALENDAR/SESSION API -> POSTGRESQL

BULLMQ SESSION GENERATOR -> CREATE UPCOMING SESSION -> RESOLVE ROSTER

17. Technical Flow - Step 3: Calendar and Schedule

A timetable recurrence is converted into concrete sessions. Holidays and rescheduling rules are applied before the session is created.

18. Technical Flow - Step 4: Attendance Opens

WORKER / AUTHORIZED FACULTY -> SESSION STATE = OPEN -> ALLOWED METHOD BECOMES ACTIVE

If Dynamic QR is enabled, the backend produces a short-lived signed token. If Manual is enabled, Faculty receives the roster. If Virtual is enabled, integration events can start arriving.

19. Technical Flow - Step 5: Attendance Capture

LEARNER/FACULTY/PROVIDER -> NEXT.JS OR WEBHOOK -> NESTJS ATTENDANCE API -> VALIDATION

The backend checks authenticated identity, tenant, enrollment, session state, allowed method, time window and security controls.

20. Technical Flow - Step 6: Evidence and Rule Processing

VALID INPUT -> SAVE RAW EVIDENCE IN POSTGRESQL -> POLICY ENGINE -> PROVISIONAL STATUS

Example: scan at 10:02 may become Present; scan at 10:16 may become Late; 45% virtual duration may become Partial depending on the tenant's versioned policy.

21. Technical Flow - Step 7: Review and Finalization

FACULTY REVIEW UI -> NESTJS ATTENDANCE SERVICE -> SUBMIT -> OPTIONAL APPROVAL -> FINALIZED

Once finalized, the official learner-session Attendance record is protected. Later changes use the correction/leave workflow and create audit history.

22. Technical Flow - Step 8: Results

FINAL RECORDS -> BULLMQ AGGREGATION -> POSTGRESQL SUMMARY / CACHE -> DASHBOARD

The system calculates subject/course attendance, group attendance, duration/compliance and configured threshold status. Redis may cache results, but PostgreSQL remains authoritative.

23. Technical Flow - Step 9: Alerts and Actions

ATTENDANCE FINALIZED/CORRECTED -> EVENT/JOB -> RULE CHECK -> NOTIFICATION SERVICE

- Learner check-in confirmation.
- Late/absence outcome.
- Low-attendance warning.
- At-risk escalation.
- Corporate compliance follow-up.
- Correction/leave decision.
- Optional make-up-session action.

SIS / HRMS / TEAMS / ZOOM / ACCESS SYSTEM -> ADAPTER -> VALIDATED EVENT -> CORE ATTENDANCE

24. Technical Flow - Step 10: Integrations

Integration adapters translate provider-specific data into common Attendance evidence. This prevents core Attendance code from becoming dependent on one external vendor.

25. Technical Flow - Step 11: AI

FINAL/AGGREGATED ATTENDANCE DATA -> AI/ANALYTICS LAYER -> RISK / SUMMARY / ANOMALY INSIGHT -> HUMAN ACTION

AI should not decide official Present/Absent status. Official status remains deterministic. AI can predict risk, summarize trends and prioritize review.

26. Final Complete Technical Flow

1. USER LOGS IN THROUGH EXISTING LMS AUTHENTICATION
2. BACKEND RESOLVES TENANT + ROLE + SCOPE
3. COURSE/GROUP + ENROLLMENT DATA DEFINES ELIGIBILITY
4. CALENDAR + TIMETABLE CREATES SCHEDULE RULES
5. BULLMQ WORKER GENERATES SESSION + ROSTER
6. SESSION LOADS VERSIONED ATTENDANCE POLICY
7. ATTENDANCE OPENS
8. MANUAL / DYNAMIC QR / GPS / VIRTUAL / OTHER EVIDENCE ARRIVES
9. NESTJS VALIDATES IDENTITY + TENANT + ENROLLMENT + SESSION + SECURITY
10. POSTGRESQL STORES RAW EVIDENCE
11. POLICY ENGINE CALCULATES PROVISIONAL STATUS
12. FACULTY REVIEWS AND SUBMITS
13. BACKEND FINALIZES OFFICIAL ATTENDANCE
14. WORKERS RECALCULATE % / DURATION / COMPLIANCE
15. REDIS CACHES SAFE HIGH-READ DATA
16. NOTIFICATIONS + REPORTS + INTEGRATIONS UPDATE
17. OPTIONAL AI PRODUCES INSIGHTS - NOT OFFICIAL ATTENDANCE DECISIONS

27. Final Technology Responsibility Map

Technology	One-line responsibility in NeuroLXP Attendance
Next.js	Creates Attendance pages/routes inside the LMS.
React	Creates interactive reusable Attendance UI components.
TypeScript	Keeps frontend/backend contracts and Attendance states type-safe.
Node.js	Runs the backend application and integration/event I/O.
NestJS	Organizes secure Attendance APIs and business modules.
PostgreSQL	Stores permanent authoritative Attendance data.
Prisma	Connects NestJS to PostgreSQL with typed models/migrations.
Redis	Handles fast temporary state, cache, rate limits and queue storage.
BullMQ	Runs asynchronous/time-based Attendance jobs.
SSO/OIDC/JWT	Provides trusted user, tenant and role identity.
S3 storage	Stores permitted Attendance-related files/exports.
OpenAPI	Documents API contracts.
Jest/Vitest	Tests services and deterministic Attendance rules.
Playwright	Tests complete Faculty/Learner browser flows.
OpenTelemetry	Observes errors, latency, queues and integrations.
Docker/CI-CD	Builds, tests and deploys the Attendance services consistently.

Important implementation note: these are recommended technologies for the Attendance architecture. Wherever NeuroLXP already has a stable equivalent - authentication, ORM, queue, notification, storage, monitoring or deployment - reuse the existing platform standard instead of creating a parallel stack.